

# Toward Accountable Intuition: A Traceable Live-Play Architecture for Bounded Machine Action on the Intellivision

Working draft

October 2026

## Abstract

This paper proposes an architecture for bounded machine action in original videogame play. An Intellivision Master Component remains the authoritative game and display machine. A separate 6x09-family computer, a future wire-wrapped HD6309 companion, and a host system observe play, preserve an explicit decision trace, and only later offer fail-neutral controller actions. We call the first target an *intuitive program*: a bounded program that identifies task-relevant differences, ranks permitted candidate actions, and revises that tendency from observed consequences. This draft does not claim general intelligence or completed learning results.

## 1 Introduction

Machine-learning systems can produce competent play while leaving the route from observation to action difficult to inspect. Historical computers can make each bit visible while never adapting their behaviour. This project asks whether an original videogame platform can support a bounded experiment in which a program selects what to attend to while leaving enough evidence for another person to reconstruct why an action was permitted.

The target game system is an original Intellivision Master Component, model 2609. It runs original game software and remains the source of displayed video. The companion observes the game externally and retains structured history. It does not begin as a second master of the console bus.

## 2 Literature survey and claimed novelty

The closest technical literature is not a single field but an intersection of programmatic reinforcement learning (RL), explainable RL, and formalized action constraints. Programmatic RL represents policies as executable programs rather than solely as opaque neural networks. Shabadi, Fijalkow, and Matricon study programmatic policies for gridworlds, including policy-size bounds and a synthesis procedure [5]. Liu et al. use an LLM-guided search to synthesize programmatic policies in Karel-like tasks [3]. These are important precedents for the proposal’s use of bounded maze policies, but they do not make a physical action boundary or live hardware provenance their central object.

Recent work also demonstrates that readable programmatic policies can be useful under constrained computation. Hu, Zhang, and Baier’s ProRL represents scheduling strategies in a domain-specific language, searches a program space, and learns parameters while retaining editable policy structure [2]. This is especially close to the present proposal’s interest in small, revisable policies. It

differs in task and substrate: scheduling is not live play against an original physical game machine, and the work does not require a human/machine authority gate at an electrical boundary.

Explainable RL provides the second neighbouring field. Saulières surveys the area through the questions of what is explained and how it is explained [4]. ASQ-IT provides interactive video-based explanations of RL behaviour using temporal-logic queries [1]. Such work is valuable for asking questions of an agent after or alongside its operation. The present proposal is narrower and more infrastructural: it attempts to make the observation, candidate set, authorization, act, verification, and fault state native records of the action system itself, rather than explanations generated solely after a policy has acted.

The claimed novelty is therefore not that a program can learn a policy, nor that an RL policy can be explained. It is an *embodied, provenance-first architecture for intuitive programs*. A candidate-ranking policy is separated from authorization to act; a physical, fault-neutral boundary and immediate human takeover constrain later controller emulation; and a replayable trace spans video observation, finite-state context, candidate ranking, authority, bounded action, verification, and outcome. The claim remains a design claim until the protocol yields live trials. It should be evaluated against the cited fields rather than asserted as an unexplored category.

### 3 Intuitive programs

An *intuitive program* identifies task-relevant differences, ranks a small set of permitted next actions, and revises that ranking from observed consequences. It is not thereby general, conscious, or entitled to unrestricted control. For a maze-oriented task, the intended loop is:

```
frame/history → relevant differences → candidates → ranked choice → bounded
               action → consequence → revised tendency.
```

The first policies may be transparent: wall following, target seeking, or bounded exploration. A learned host policy may later rank the same candidates. A ranking is evidence for a choice, not authority to change hardware state.

### 4 Accountable action

The design makes five commitments:

1. A difference becomes a signal only when it can alter a permitted decision.
2. Comparison is evidence, not an implicit state write.
3. A candidate is not a physical action until authority and guard conditions have passed.
4. An act is not established until it has been verified and traced.
5. Ambiguity, timeout, contradiction, and low confidence produce a named pause, neutral, or human-takeover state.

The trace is an Ariadne thread: an ordered account of state, candidate, selection, action, result, error, or halt. It makes a finite controller open to inspection without confusing that controller with proof of general intelligence.

## 5 Apparatus

The Intellivision 2609 runs the original game and must remain functional when all companion components are absent, unpowered, reset, or faulted. The M6x09-I SBC is the operational 6x09 reference platform for RAM-loaded programs, serial traces, timing experiments, and passive daughterboard work; its RS-232 monitor path is verified at 19,200 8N1. The wire-wrapped HD6309 board is the primary future artifact and transfers only demonstrated behaviour from the reference platform. A host computer captures video, archives trials, and optionally evaluates candidate-ranking policies. A human player retains immediate takeover authority.

The M6x09-II SBC remains a useful diagnostic/design reference, but its ACIA serial acceptance is unresolved and it does not gate this experimental path.

## 6 Decision and authority contract

Each machine turn is a transaction:

`target → candidate → compare → guard → bounded action → verify → next state  
or halt.`

State, timing, fault, and human/machine ownership guards decide whether a candidate is lawful. A physical selector exposes Human, Neutral, and Machine states. Reset, watchdog timeout, host loss, or fault must return a future controller boundary to disabled/neutral. The first experiments contain no controller injection: they use external video capture and timestamped human input. Controller emulation is later work following direct electrical measurement.

## 7 Trace, evidence, and agent guidance

Each trial records frame identifiers or extracted features; current state and permitted candidates; candidate ranking and selected action; policy and firmware versions; authority source; duration; outcome; and error, timeout, neutral, or takeover transitions. Score is not the sole measure of success.

An agent serves as an engineering shadow, not as a source of hardware truth. It may organize evidence, cross-check sources, state hypotheses, and propose the smallest discriminating test. Each material step preserves the observation and conditions, alternatives, human decision, next test, and correction. This supports a later podcast account without allowing an agent explanation to become untested electrical evidence.

## 8 Path to empirical results

Before this manuscript reports performance or adaptation, the project requires:

1. a preserved M6x09-I S-record load and RAM-execution transcript;
2. a fixed maze-oriented game, configuration, start state, and action set;
3. a transparent baseline policy;
4. replayable trials using the stated trace format;

5. a human or fixed-policy comparison; and
6. evidence that candidate rankings changed after outcomes.

Until then, the contribution is an architecture and protocol for testing intuitive programs honestly. Once these materials exist, this draft can become an arXiv-ready empirical report with reproducible artifacts, figures, related work, and an explicit claim audit.

## 9 Conclusion

The intended threshold is modest but substantive: a machine begins to select what to attend to and what to try from experience, while another person can still inspect how and why.

## Contextual materials

- The Last Cyberneticist, “[A Machine Must Know Its Next Move](#),” 2026.
- The Last Cyberneticist, “[From Algebra to Algorithm: Multenions and Program Synthesis](#),” 2026.

## References

- [1] Yotam Amitai, Ofra Amir, and Guy Avni. Interactive explanations for reinforcement-learning agents. *arXiv preprint arXiv:2504.05393*, 2025.
- [2] Chengpeng Hu, Yingqian Zhang, and Hendrik Baier. Scheduling that speaks: An interpretable programmatic reinforcement learning framework. *arXiv preprint arXiv:2605.18454*, 2026.
- [3] Max Liu, Chan-Hung Yu, Wei-Hsu Lee, Cheng-Wei Hung, Yen-Chun Chen, and Shao-Hua Sun. Synthesizing programmatic reinforcement learning policies with large language model guided search. *arXiv preprint arXiv:2405.16450*, 2024.
- [4] Léo Saulières. A survey of explainable reinforcement learning: Targets, methods and needs. *arXiv preprint arXiv:2507.12599*, 2025.
- [5] Guruprerana Shabadi, Nathanaël Fijalkow, and Théo Matricon. Programmatic reinforcement learning: Navigating gridworlds. *arXiv preprint arXiv:2402.11650*, 2024.